```
  "devDependencies": {
    "jasmine": "^2.9.0",
    "jasmine-core": "^2.9.1",
    "karma": "^2.0.0",
    "karma-chrome-launcher": "^2.2.0",
    "karma-jasmine": "^1.1.1",
    "karma-mocha-reporter": "^2.2.5",
    "karma-sourcemap-loader": "^0.3.7",
    "karma-webpack": "^2.0.9",
    "testdouble": "^3.3.3",
    "testdouble-jasmine": "^0.2.1",
    "webpack-dev-server": "^2.11.1"
  }
}
```

That scripts bit at the top will let you run the tests using the command yarn test. The command itself includes NODE_ENV=test to force Webpack to use the test.js version of the configuration.

Okay, everybody still here? It's time to write a sample test.

Writing a Sample Test

This first test is just here to introduce Jasmine and Karma, not to be any kind of useful test. This is Jasmine, using ES6 syntax, mostly the arrows:

js_jasmine/01/spec/javascripts/basic_spec.js

```
describe("JavaScript testing", () => {
  it("works", () => {
    expect(1 + 1).toEqual(2)
  })
})
```

Jasmine's syntax is heavily inspired by RSpec, with JavaScript function objects taking the place of Ruby blocks. Like RSpec test suites, Jasmine test suites start with describe. In Jasmine, describe is a method that takes two arguments: a string and a function. The function defines the behavior of an entire test suite.

Inside the describe function argument, individual specs are defined by calls to the function it, which also takes a string description argument and a function argument. The function argument to it is the actual spec.

Inside the spec, individual expectations are denoted with the expect method. That method takes an argument, which should be the actual value from the test. The return value of expect is an object that Jasmine calls an *expectation*. The expectation can then be called with one of several matchers, such as